Universidade do Algarve

12 de abril de 2026

Trabalho 2:

Instalação Robótica

Tecnologias de Mercado 2025-2026

Alexandre Pita e Daniel Clímaco

Índice

Enunciado 2
Introdução 2
Instalação 2
Entidades 3
Robôs 4
Estados 4
Carregamento 4
Realizar tarefas 4
Comportamento 5
Algoritmo de busca 6
Tarefas 6
Entrada de dados 7
Comandos 9
help 9
step 9
add-task 11
get-robot 11
exit 12
Requisitos 12
Notas 12
Entrega, avaliação e validação 13
Entrega 13
Validação 13
Resumo 14
Critérios de avaliação 14

Enunciado

Introdução

O objetivo deste trabalho é desenvolver uma instalação industrial virtual em Java, automatizada por robôs que se deslocam autonomamente pela instalação. O utilizador interage com o programa através da consola, sendo o respetivo *output* apresentado na mesma.

Os robôs realizam as tarefas que o utilizador fornecer. O programa deve rodar uma simulação que faz os robôs pegarem numa tarefa e a realizarem. As tarefas consistem em transportar um dado objeto para um dado destino. A simulação corre em passos/iterações (*steps*) e apenas corre a próxima quando assim for pedido pelo utilizador. O utilizador interage com o programa através da consola usando comandos. Por exemplo, para executar a próxima iteração executaria o comando "step".

Instalação

A instalação é representada por um mapa retangular finito, dividido em células organizadas em grelha. Ao imprimir o mapa da instalação, deve ser apresentado um retângulo composto por caracteres com os seguintes significados.

Representação	**Significado**
`..`	Célula vazia
`##`	Um obstáculo
`RN`	Um robô com ID *N*
`RR`	Múltiplos robôs na célula
`ON`	Objeto com ID *N*
`RN`	Um robô com ID *N* e um objeto
`SN`	Estação de carregamento do robô com ID *N*
`CN`	Estação de carregamento com o robô *N* a restaurar a carga

Observações:

- Cada célula ocupa 2 caracteres e entre cada uma há 1 espaço em branco.
- Para os robôs, objetos e estações de carregamento, deve ter a letra correspondente e o ID (por exemplo R2 seria robô com ID 2).
- Se numa dada célula estiver mais do que 1 robô, esta deve ser representada por `RR`.

- Se um robô estiver na sua estação de carregamento a restaurar a carga, a estação deve ser representada pela letra “C” mais o ID do robô, caso contrário deve ser “S” mais o ID do robô a que esta estação pertence.

Além disso, cada linha e coluna deve ser representa por um número, que representa a coordenada.

Exemplo:

```
   01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
01 .. .. .. .. .. .. .. .. .. .. .. O2 .. ## .. .. .. ..
02 O3 .. ## ## ## .. O1 .. .. .. .. .. .. ## .. .. .. ..
03 .. .. ## .. .. .. .. .. .. .. .. .. .. ## .. .. ## ..
04 ## .. .. .. .. .. .. ## .. .. ## .. .. .. .. .. ## ..
05 ## .. .. .. .. .. .. ## .. .. ## ## ## ## ## .. R2 ..
06 .. .. ## .. S1 .. .. ## .. .. .. .. .. .. ## .. .. ..
07 .. .. ## .. .. .. .. ## ## ## ## ## .. .. ## .. S2 ..
08 .. R1 ## .. .. .. .. .. .. .. .. .. .. .. ## .. .. ..
```

O exemplo mostra o mapa de uma instalação com dimensões 18 por 8.

Neste sistema, um par de coordenadas é representado pelo número da linha e número da coluna. Por exemplo, o robô R1 está em `12;4` e o robô R2 em `7;7`. Coordenadas devem ser representadas nesse formato, em que o primeiro número representa a coluna (coordenada x) e o segundo a linha (coordenada y).

Entidades

Uma entidade é algo presente no mapa. Existem 4 tipos de entidade:

- **Obstáculo** – Um objeto ou estrutura inamovível.
- **Robô** – Executa tarefas de transportar objetos.
- **Estação de Carregamento** – Local onde o robô correspondente pode restaurar o seu nível de carga.
- **Objeto** – Componente da instalação industrial que pode ser transportado pelos robôs.

Todos os robôs e objetos têm um ID representado por um número inteiro. Estes IDs são obtidos pela ordem pelos quais são especificados nos dados iniciais, começando em 1. Robôs e objetos têm sequências separadas de IDs.

Robôs

O objetivo de cada robô é realizar tarefas fornecidas pelo utilizador, durante as quais terá de se deslocar pela instalação. Cada robô tem um certo nível de carga, o qual diminui a cada iteração, consoante a seguinte tabela.

Estado	**Carga por iteração**
Parado fora da estação de carregamento	-0,2%
Parado na estação de carregamento	+0,0%
Mover 1 célula	-0,5%
A restaurar carga	+10,0%

Estados

Os robôs podem estar em 1 de 3 estados:

- `IDLE` – Parado e disponível para realizar alguma tarefa. Vai verificando as tarefas disponíveis e pegar na primeira que conseguir realizar.
- `BUSY` – Ocupado a realizar uma tarefa.
- `CHARGING` – Numa estação de carregamento a restaurar a sua carga.

Carregamento

Cada robô deve ter a sua própria estação de carregamento, que, quando nela presente, irá restaurar a carga do robô consoante a tabela anterior. Cada robô só pode usar a sua própria estação de carregamento e não a de outros robôs. Quando um robô se encontra na estação de carregamento, não deve sair dela nem deve aceitar novas tarefas até a sua carga atingir os 100%. Após atingir 100% de carga, o robô passa do estado `CHARGING` para o estado `IDLE` e, enquanto se mantém na estação de carregamento, a sua carga mantém-se a 100%.

Realizar tarefas

O utilizador não pode controlar diretamente um robô, mas sim adicionar tarefas à simulação para este as realizar. Em cada iteração da simulação, se o robô não tiver nenhuma tarefa associada nem estiver a restaurar a sua carga, este irá consultar a lista de tarefas (pela ordem da mais antiga para a mais recentemente adicionada) e pegar na primeira tarefa livre que conseguir realizar.

Antes de um robô aceitar e executar uma tarefa, este tem de verificar se consegue realizá-la. O robô deve internamente calcular os caminhos que terá de fazer para realizar a tarefa e voltar à estação de carregamento. Os caminhos são:

1. Ir até ao objeto (para pegar no objeto)
2. Ir até ao destino (para colocar lá o objeto)
3. Ir até à estação de carregamento (caso, após terminar a tarefa, não tenha carga suficiente para realizar outra tarefa)

Comportamento

- Depois do robô chegar ao objeto, este deve **pegar nele e deslocar-se com ele até ao destino** (definido na tarefa). Enquanto o objeto estiver a ser transportado pelo robô, o mesmo não deve ficar visível no mapa da instalação.
- Quando o robô chegar ao destino, deve **colocar o objeto nessa célula**. Lembrando que, enquanto o robô estiver na mesma célula que o objeto, no mapa apenas fica visível o robô. Assim que o robô sair dessa célula, o objeto fica visível.
- Depois do robô ter chegado ao destino e ter colocado o objeto, **a tarefa toma-se como concluída**. Depois, **na mesma iteração**, o robô irá consultar a lista de tarefas e verificar se consegue realizar alguma das tarefas atualmente disponíveis. Se encontrar uma tarefa que consiga realizar, deve aceitá-la e realizá-la, caso contrário deve-se deslocar até à sua estação de carregamento para restaurar a carga.
- Note que, para o cálculo do custo de uma tarefa, mesmo que após a sua realização o robô não se desloque até à estação de carregamento (por ter encontrado uma tarefa que consegue realizar), deve sempre calcular e considerar o custo do caminho até à estação. Isto porque, até concluir a tarefa, o robô não sabe se existirá alguma tarefa adicional que consiga realizar; caso contrário, terá de se deslocar até à estação. Resumidamente, deve assumir o pior caso (aquele em que consume mais carga).
- O robô não deve aceitar nenhuma tarefa durante todo o período em que se estiver a deslocar até à estação nem enquanto lá estiver a restaurar a carga. Assim que a carga atingir os 100%, poderá voltar a aceitar novas tarefas.
- Os três caminhos que o robô irá realizar, ou poderá vir a realizar, para uma dada tarefa são calculados através do algoritmo descrito mais à frente. Com base nesses caminhos, o robô deverá determinar se possui carga suficiente para os percorrer. Um robô nunca deve atingir um nível de carga nula, exceto no caso de ter acabado de chegar à sua estação de carregamento.

Algoritmo de busca

Os robôs devem usar o algoritmo de busca em profundidade para determinar o caminho necessário para se deslocarem até a um certo ponto na instalação. Este algoritmo deve ser aplicado antes do robô aceitar a tarefa, para calcular os caminhos e determinar se tem carga suficiente para os poder realizar.

Cada robô pode se deslocar 1 célula de cada vez em uma de 4 direções – esquerda, cima, direita e baixo. Será necessária uma lista `visited` para guardar todas as células que foram visitadas e outra lista `path` que no fim conterá o caminho até ao objetivo. O algoritmo guarda a célula ativa e realiza os seguintes passos:

1. Verifica a célula de cada uma das direções na **ordem esquerda-cima-direita-baixo** e determina a primeira que for acessível. Uma célula é acessível se todas as seguintes condições forem verdadeiras:
    - Não tem nenhum obstáculo, estação de carregamento ou objeto na célula, exceto o objeto que deve ser transportado (definido pela tarefa).
    - A célula ainda não foi visitada pelo robô, ou seja, a célula não se encontra em `visited`.
2. Com base no passo anterior:
    a) Se encontrar uma célula acessível, define esta como a célula atual, adiciona ela a `visited` e a `path`, e volta para o passo 1.
    b) Se nenhuma das 4 células for acessível, será feito *backtracking*, que consiste em definir a célula atual como a última célula que o robô visitou, retirar a última célula de `path` e voltar para o passo 1.

Se a um dado momento nenhuma das direções for válida e o `path` ficar vazio, isto indica que o caminho é impossível. Nesse caso o robô deve rejeitar a tarefa.

Tarefas

Tarefas são pedidos do utilizador, para os robôs transportarem um dado objeto para um dado destino.

Entrada de dados

Assim que o programa iniciar, este deve esperar por input de 2 coisas.

Primeiro, deve receber as dimensões da instalação no formato `W H`, em que `W` é a largura e `H` é a altura. Por exemplo `18 8`.

Depois, deve receber uma lista de obstáculos, robôs com as devidas estacões de carregamento, e objetos, um por linha, nos seguintes formatos:

- Obstáculo: `OBS` *`x y`*
  - *`x`* – coordenada *x* do obstáculo
  - *`y`* – coordenada *y* do obstáculo
- Robô: `ROB` *`x y cs-x cs-y`*
  - *`x`* – coordenada *x* do robô
  - *`y`* – coordenada *y* do robô
  - *`cs-x`* – coordenada *x* da estação de carregamento do robô
  - *`cs-y`* – coordenada *y* da estação de carregamento do robô
- Objeto: `OBJ` *`x y`*
  - *`x`* – coordenada *x* do objeto
  - *`y`* – coordenada *y* do objeto

Por fim, para indicar o fim de dados iniciais e para iniciar a simulação, o programa deve receber o texto "start".

Exemplo de dados iniciais:

```
7 5
OBS 2 3
OBS 2 4
OBS 3 4
OBS 6 2
ROB 1 4 1 2
OBJ 4 3
start
```

O programa deve validar os dados recebidos e imprimir os devidos erros caso sejam inválidos:

Descrição	**Exemplo**	
	Input	**Output**
Dimensões excedem o mínimo de 5 por 5.	`3 7`	`Minimum dimensions allowed is 5 by 5.`
Dimensões excedem o máximo de 99 por 99.	`120 70`	`Maximum dimensions allowed is 99 by 99.`
Formato inválido das dimensões.	`18,8`	`Invalid dimensions format.`
Tipo de entidade desconhecida. Entidades permitidas são "OBS", "ROB" e "OBJ".	`ABC 15 4`	`Unknown entity type "ABC" at entity 1.`
Coordenadas da entidade inválidas. As coordenadas não devem ser inferiores a 1, não devem ser superiores ao tamanho da instalação, nem devem coincidir com outra entidade.	`OBJ 130 80`	`Invalid entity coordinates at entity 1.`
Formato da entidade inválido. A entidade deve receber as coordenadas necessárias	`OBJ 10`	`Invalid entity format at entity 1.`
Não foram fornecidas nenhumas entidades. É necessária pelo menos 1 entidade.	`18 8 start`	`Expected entity name.`
Não foram fornecidos robôs ou objetos suficientes. É necessário pelo menos 1 robô e 1 objeto.	`18 8 OBJ 15 6 start`	`At least 1 robot and 1 object are required.`

Nas mensagens de erro que dizem "at entity 1", o número "1" deve ser o número da entidade. A primeira entidade é a 1, a segunda é a 2, a terceira é a 3, etc. Este número **não** está relacionado com o ID da entidade.

Após estes dados iniciais, o programa deve iniciar a simulação e apresentar o mapa da instalação. A partir deste momento, o programa deve receber comandos do utilizador.

Comandos

O utilizador interage com o programa através de comandos. O programa deve apresentar a mensagem “Enter command: ” e, de seguida, permitir que o utilizador introduza o comando.

help

Imprime a lista de comandos da seguinte forma:

```
[Help – List of commands]
help - Show list of commands
step – Run next simulation step
add-task <obj-id> <dest-x> <dest-y> – Add new task
get-robot <id> - Get robot information
exit – Exit the simulation
```

step

Corre a próxima iteração da simulação e imprime o estado atual da simulação, que deve conter a contagem de iterações (*steps*), o mapa da instalação, a lista de tarefas e a lista de robôs.

Por exemplo, no início será:

```
Step count: 0
   01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
01 .. .. .. .. .. .. .. .. .. .. .. O2 .. ## .. .. .. ..
02 O3 .. ## ## ## .. O1 .. .. .. .. .. .. ## .. .. .. ..
03 .. .. ## .. .. .. .. .. .. .. .. .. .. ## .. .. ## ..
04 ## .. .. .. .. .. .. ## .. .. ## .. .. .. .. .. ## ..
05 ## .. .. .. .. .. .. ## .. .. ## ## ## ## ## .. R2 ..
06 .. .. ## .. S1 .. .. ## .. .. .. .. .. .. ## .. .. ..
07 .. .. ## .. .. .. .. ## ## ## ## ## .. .. ## .. S2 ..
08 .. R1 ## .. .. .. .. .. .. .. .. .. .. .. ## .. .. ..

ROBOTS
[R1] pos=2;8 charge=100.00% state=IDLE task=NONE
[R2] pos=17;5 charge=100.00% state=IDLE task=NONE

TASKS

Enter command:
```

Exemplo na iteração 10:

```
Step count: 10
   01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
01 .. R1 .. .. .. .. .. .. .. .. .. O2 .. ## .. .. .. ..
02 O3 .. ## ## ## .. O1 .. .. .. .. .. .. ## .. .. .. ..
03 .. .. ## .. .. .. .. .. .. .. .. .. .. ## .. .. ## ..
04 ## .. .. .. .. .. .. ## .. .. ## .. .. .. .. .. ## ..
05 ## .. .. .. .. .. .. ## .. .. ## ## ## ## ## .. R2 ..
06 .. .. ## .. S1 .. .. ## .. .. .. .. .. .. ## .. .. ..
07 .. .. ## .. .. .. .. ## ## ## ## ## .. .. ## .. S2 ..
08 .. .. ## .. .. .. .. .. .. .. .. .. .. .. ## .. .. ..

ROBOTS
[R1] pos=2;1 charge=97.70% state=BUSY GOING TO OBJECT (5/14) task=(O1 --> 10;5)
[R2] pos=17;5 charge=98.00% state=IDLE task=NONE

TASKS
(O1 --> 10;5) - ASSIGNED TO R1

Enter command:
```

Explicação dos dados do robô R1:

- `[R1]` – "1" significa ID 1 do robô R1
- `pos=2;1` – Posição atual do robô (x é 2, y é 1)
- `charge=99.80%` - Carga atual do robô (arredondado a 2 casas decimais)
- `state=BUSY GOING TO OBJECT (5/14)` – Atualmente o robô encontra-se no estado `BUSY` e está no caminho até ao objeto, mais concretamente na posição 5 de 14 do caminho
- `task=(O1 --> 10;5)` – Atualmente o robô está assignado à tarefa de transportar o objeto O1 para 10; 5.

Possíveis estados de um robô:

Descrição	**Representação**
Parado sem tarefa	`IDLE`
Ocupado a ir até ao objeto	`BUSY GOING TO OBJECT (P/N)`
Ocupado a ir até ao destino	`BUSY GOING TO DESTINATION (P/N)`
Ocupado a ir até à estação de carregamento	`BUSY GOING TO CHARGING STATION (P/N)`
A restaurar carga	`CHARGING`

- `P` – Posição atual (começa em 1 e vai até N)
- `N` – Comprimento do caminho (número de posições total)

add-task

Cria uma nova tarefa e adiciona-a à lista de tarefas.

Sintaxe:

```
add-task <obj-id> <dest-x> <dest-y>
```

Pede ao robô para transportar o objeto com ID `obj-id` para a célula destino (`dest-x`; `dest-y`).

Validações:

- Se for especificado um ID que não existe deve apresentar a mensagem "Object with ID X not found.", em que X é o ID especificado.
- Se forem especificadas coordenadas inválidas (fora do mapa ou numa célula não vazia) deve apresentar a mensagem "Invalid destination.".
- Se o objeto já estiver associado a uma tarefa deve apresentar a mensagem "Object already associated to a task.".

get-robot

Sintaxe:

```
get-robot <id>
```

Descrição:

Imprime informações sobre o robô:

- ID
- Nível de carga
- Estado
- Tarefa atual

Exemplos:

```
[R1]  pos=2;8 charge=100.80% state=IDLE task=NONE

[R1]  pos=1;5 charge=82.20% state=BUSY (GOING TO OBJECT) task=(O1 -->
10;4)

[R1]  pos=6;1 charge=64.40% state=BUSY (GOING TO DESTINATION) task=(O1 --
> 10;5)

[R1]  pos=9;4 charge=32.40% state=BUSY (GOING TO CHARGING STATION)
task=NONE
```

Validações:

- Se for especificado um ID que não existe deve apresentar a mensagem "Robot with ID X not found.", em que X é o ID especificado.

exit

Termina o programa.

Requisitos

- O trabalho deve ser implementado em Java, com interação do utilizador através da consola.
- Deve utilizar apenas os conhecimentos e conteúdos lecionados nas aulas.
- Pode fazer uso de estruturas de dados como *ArrayList*, *LinkedList* e *HashMap*.
- O código deve seguir boas práticas de programação orientada a objetos, utilizando conceitos como classes, interfaces, herança, encapsulamento e polimorfismo, tal como dado nas aulas.
- A implementação deve ser modular, organizada e de fácil manutenção.
- A classe *Main* deve conter apenas a leitura dos dados iniciais e inicialização da simulação.
- O *output* do programa tem de ser exatamente da forma como descrito no enunciado e como está nos exemplos.
- **Todas as classes e todos os métodos principais devem conter comentários *Javadoc*.** Os de classes devem conter pelo menos a **descrição da classe**. Os de métodos devem conter pelo menos a **descrição do método** e, quando aplicável, os **parâmetros e retorno**.

Notas

- Adicione todas as classes, atributos e métodos que achar necessários.
- Adicione comentários onde achar necessário, para ajuda própria e para quem for ler o código.
- É aconselhável começar por identificar e implementar as funcionalidades principais, adicionando as restantes de forma incremental.
- Ao longo de todo o desenvolvimento do trabalho, tire partido do *Mooshak* para validar se o programa está corretamente implementado.

Entrega, avaliação e validação

Este trabalho recebe *input* e imprime *output* pela consola. O output deve ser exatamente como é pedido no enunciado.

Entrega

A entrega do trabalho deve ser feita pela plataforma *Mooshak* de Tecnologias de Mercado. Esta plataforma irá correr diversos testes para validar se o programa foi implementado como pedido. Deve fazer *login* no Mooshak usando o concurso "Trabalho2" e usando as credenciais que lhe foram enviadas por e-mail.

Pode submeter o trabalho no *Mooshak* as vezes que desejar, de modo a verificar se passa os testes necessários. O seu objetivo é obter o maior número de testes aceites até ao prazo final de entrega. Idealmente, deve passar em 100% dos testes; no entanto, mesmo que isso não aconteça, o trabalho será aceite para avaliação e validação. Na tutoria, encontra-se disponível uma pasta comprimida (.zip) com ficheiros contendo os inputs e outputs esperados de todos os testes do *Mooshak*. Consulte esses ficheiros para identificar a razão pela qual algum teste falhou. É também importante notar que, se tentar submeter o trabalho sem ter implementado o "start" ou o "exit", vários testes irão falhar com erro de execução. Nesses casos, o programa não irá terminar (pois permanece continuamente à espera de dados), e o *Mooshak* considera-o como um programa inválido. **Deve consultar o documento Plataforma Mooshak, presente na tutoria, para saber mais sobre a plataforma e como a usar.**

O prazo final de entrega é até dia **6 de maio às 21:00**. **Não são aceites entregas após o prazo**, salvo ocasiões com justificação válida e aceite pelo docente.

Validação

Após a entrega, o grupo deve realizar a validação do trabalho com o professor no dia 13 de maio durante a aula das 9:00 às 13:00. A validação é obrigatória e a não realização da mesma resulta em nota nula no trabalho. Para que o trabalho seja elegível para validação, **tem de ter pelo menos 150 pontos** no *Mooshak*. A pontuação máxima no *Mooshak* é de 1000 pontos, o que equivale a passar a todos os 20 testes.

A validação consiste numa discussão sobre o trabalho, entre o grupo e o professor, para validar a autoria do mesmo, e avaliar a compreensão técnica do que foi desenvolvido. Durante a discussão, os alunos deverão saber explicar o funcionamento do código e as decisões tomadas. Qualquer um dos alunos, independentemente de qual tenha implementado determinada parte do código, **deve ser capaz de explicar, pelo menos, a ideia geral dessas partes**. Também poderá ser exigido aos alunos implementarem ou alterarem algo no código. Esta etapa poderá impactar a nota final do trabalho.

Resumo

Prazo de entrega: 6 de maio às 21:00

Modo de entrega: *Mooshak* – Concurso "Trabalho2"

Pontuação mínima no Mooshak para validação: 150 pontos

Pontuação máxima no Mooshak: 1000 pontos

Data da validação: 13 de maio das 9:00 às 13:00

Critérios de avaliação

A avaliação do trabalho terá em conta, além de outros fatores, os seguintes principais critérios:

- Número de pontos no *Mooshak*.
- Implementação correta das funcionalidades pedidas.
- Cumprimento dos requisitos citados.
- Qualidade do código e a utilização adequada de boas práticas e conceitos de programação orientada a objetos.
- A organização e modularidade do código.

Com base nestes critérios, os professores atribuirão a nota que acharem justa, **a cada elemento do grupo individualmente**. A implementação de funcionalidades não pedidas não é compensada nem afeta a nota do trabalho. O trabalho deve ser apenas realizado pelos elementos constituintes do grupo.

Aviso importante

A inclusão de partes significativas no trabalho, provenientes de código desenvolvido por elementos não pertencentes ao grupo ou gerado por inteligência artificial, é considerada desonestidade académica e pode resultar na anulação do trabalho ou na reprovação do aluno na cadeira.

Bom trabalho!